PostgreSQL

Function - Trigger
Functional Dependency - Foreign Key - Index

Nguyễn Văn Diêu

Ho Chi Minh City University of Transport

2026

Kiến thức - Kỹ năng - Sáng tạo - Hội nhập

Sứ mệnh - Tầm nhìn

Triết lý Giáo dục - Giá trị cốt lõi

Outline I

1. PL/pgSQL Function

1.1 Syntax

1.2 Declearing variables

1.3 Copy Datatype

1.4 Return Record

1.5 Return Query Results

- GPA for each student
- Quadratic equation

1.6 Case Conditional Expressions

1.7 Loop

1.8 For Loop

1.9 Array Loop

2. PL/pgSQL Trigger Function

2.1 Types of Trigger

2.2 Variables passed to the TRIGGER function

Outline II

2.3 Create Trigger
2.4 Drop Trigger
2.5 Truncate command
2.6 Audit Trigger
2.7 Disallowing DELETE
2.8 Disallowing TRUNCATE
2.9 Disallowing TRUNCATE
2.10 Conditional Triggers
2.11 Integrity Constraints Form
- function
- trigger
- e.g. grades

3. Functional Dependency
3.1 $R(ABC)$, $A \rightarrow B$
- insert R
- update A or B of R

Outline III

3.2 $R1(ABC), R2(CDE),\ B \rightarrow D$
- insert $R1$
- update B or C
- insert $R2$
- update C or D

4. Foreign Key

5. Index

5.1 Create index

5.2 Drop index

5.3 Index types

5.4 UNIQUE index

5.5 Index on Expression

5.6 Multicolumn indexes

Syntax Function

```
create [or replace] function func_name(p1 type, p2 type, ...)
returns ret_type
language plpgsql
as
$$
declare
    - - variable declaration [:= expression];
begin
    - - logic ;
end ;
$$
```

Declare

```
do
$$
declare
   x integer;
   y date := '2022-11-07';
   z timestamp;
begin
   x := 1;
   z := now();
   raise notice 'var: x = %, y = %, z = %', x, y, z ;
end
$$
```

Declare in Function

```
create or replace function sumxy(integer, integer)
returns integer
language plpgsql
as
$$
declare
  ret integer;
begin
  ret := $1 + $2;
  return ret;
end
$$
```

call:

```
select sumxy(2, 3)
```

Declare via alias for

```
create or replace function sumxy(integer, integer)
returns integer
language plpgsql
as
$$
declare
  ret integer;
  x alias for $1;
  y alias for $2;
begin
  ret := x + y;
  return ret;
end
$$
```

call: **select sumxy(2, 3)**

Copy Datatype

```
do
$$
declare
   x subject.units%type;
begin
   select units into x
   from subject
   where subjectid = 's01';

   raise notice 'x = %', x;
end;
$$
```

Return Record

```
do
$$
declare
   st subject.subjectname%type ;
begin
   select subjectname into st
   from subject
   where subjectid = 's01' ;

   raise notice 'st = %', st ;
end ;
$$
```

Return Query Results

```
create or replace function st_gpa()
returns table (stid char(3), gpa float)
language plpgsql as
$$
begin
   return query
   select studentid stid, avg(grades) gpa
   from studentgrades
   group by studentid
   order by studentid ;
end ;
$$
```

call:

```
select * from st_gpa()
```

e.g. Quadratic equation

```
create or replace function quad_eq (a float, b float, c float)
returns table(x1 float, x2 float)
language plpgsql as
$$
declare
  delta float;
  x1 float;
  x2 float;
begin
  delta := b*b - 4*a*c;
  if delta < 0 then
    return query
    select x1, x2 where 1 = 2 ;
```

e.g. Quadratic equation

```
    elseif delta = 0 then
      x1 := -b / (2*a) ;
      return query
      select round(x1::decimal,2)::float , round(x1::decimal,2)::float ;
    else
      x1 := (-b - sqrt(delta)) / (2*a);
      x2 := (-b + sqrt(delta)) / (2*a);
      return query
      select round(x1::decimal,2)::float , round(x2::decimal,2)::float ;
    end if;
end
$$
call:
      select quad_eq(1,1,1);
      select quad_eq(1,-2,1);
      select quad_eq(1,-3,1);
```

using return test

```
create or replace function quad_eq2 (a float, b float, c float)
returns text
language plpgsql as
$$
declare
   mess text;
   delta float;
   x1 float;
   x2 float;
begin
   delta := b*b - 4*a*c;
   if delta < 0 then
      select 'No solution' into mess ;
```

using return text

```
    elseif delta = 0 then
        x1 := -b / (2*a) ;
        select 'x1 = x2 = ' || round(x1::decimal,2)::float into
mess ;
    else
        x1 := (-b - sqrt(delta)) / (2*a);
        x2 := (-b + sqrt(delta)) / (2*a);
        select 'x1 = '|| round(x1::decimal,2)::float ||
            ' ; x2 = '|| round(x2::decimal,2)::float into mess ;
    end if;
    return mess ;
end
$$
```

Case

```
case when condition then result
    [ when ... ]
    [ else result ]
end
```

e.g.

```
select a,
        case when a = 1 then 'one'
            WHEN a = 2 then 'two'
            else 'other'
        end
from test;
```

Loop

```
create or replace function fib(n integer)
returns integer
as
$$
declare
    counter integer := 0;
    a integer := 0;
    b integer := 1;
begin
    if (n < 1) then
        return 0;
    end if;
    loop exit when counter = n;
        counter := counter + 1;
        select b, a + b into a, b;
```

Loop (con.)

```
    end loop;
    return a;
end;
$$
language plpgsql;
```

e.g. **select** fib(4);

For Loop

```
create or replace function fib(n integer)
returns integer as
$$
declare
    counter integer := 0; a integer := 0; b integer := 1;
begin
    if (n < 1) then
        return 0;
    end if;
    for counter in 1..n loop
        select b, a + b into a, b;
    end loop;
    return a;
end;
$$
language plpgsql;
```

Array Loop

```
create function findmax(int[ ]) returns int8
as
$$
declare max int8 := 0; x int;
begin
    foreach x in array $1 loop
        if x > max then
            max := x;
        end if;
    end loop;
    return max;
end;
$$
language plpgsql;
```

PostgreSQL Trigger

Event in table:

- **insert**
- **update**
- **delete**
- **truncate**

Automatically, trigger is invoked an event associated with a table.

In **PostgreSQL**, a trigger is defined in two steps:

- Define a trigger function using **create function**.
- Bind this trigger function to a table using **create trigger**.

Types of Trigger

1. **Row Level Trigger**: Trigger function call **for each row** and in that getting modified by the event.

   e.g. If we **update** 100 rows in the table, the **update trigger** will be called 100 times, once for each updated row.
2. **Statement Level Trigger**: Trigger function call **for each statement** and the trigger function run only once for each statement.

Variables return

OLD: Record; **Update, Delete**.
NEW: Record; **Insert, Update**.
TG_NAME: name; Denotes the name of the trigger.
TG_WHEN: text; BEFORE, AFTER, or INSTEAD OF.
TG_LEVEL: text; ROW or STATEMENT.
TG_OP: text; INSERT, UPDATE, DELETE, or TRUNCATE.
TG_TABLE_NAME: name; The name of the table.
TG_TABLE_SCHEMA: name; The schema name of the table.

Create Trigger

CREATE TRIGGER **trigger-name**

{ BEFORE | AFTER | INSTEAD OF } { **event** [OR ...] }

ON **table-name**

[FOR [EACH] { ROW | STATEMENT }]

EXECUTE PROCEDURE **function-name (arguments)**

—

event: **INSERT, UPDATE, DELETE, TRUNCATE**

e.g.

```
create or replace function func_notify_trigger()
  returns trigger
  language plpgsql
  as $$
begin
  raise notice 'Hi, I got % invoked FOR % % % on %' ,
    TG_NAME ,
    TG_LEVEL ,
    TG_WHEN ,
    TG_OP ,
    TG_TABLE_NAME ;
  return new ;
end ;
$$
```

e.g.

```
create or replace trigger notify_insert_trigger
   after insert on subject
   for each row
   execute procedure func_notify_trigger() ;
```

—

call:

```
insert into subject values('s10', 'xxx', 3)
```

—

result:

```
NOTICE: Hi, I got notify_insert_trigger invoked FOR ROW
AFTER INSERT on subject

INSERT 0 1
```

Create Trigger

Using the same function for trigger event: **UPDATE** and **DELETE**

```
CREATE OR REPLACE TRIGGER notify_update_trigger
  AFTER UPDATE ON subject
  FOR EACH ROW
  EXECUTE PROCEDURE func_notify_trigger() ;

CREATE OR REPLACE TRIGGER notify_delete_trigger
  AFTER DELETE ON subject
  FOR EACH ROW
  EXECUTE PROCEDURE func_notify_trigger() ;

CREATE OR REPLACE TRIGGER notify_trigger
  AFTER INSERT OR UPDATE OR DELETE ON subject
  FOR EACH ROW
  EXECUTE PROCEDURE func_notify_trigger() ;
```

Drop Trigger

DROP TRIGGER notify_insert_trigger ON notify_test ;

DROP TRIGGER notify_update_trigger ON notify_test ;

DROP TRIGGER notify_delete_trigger ON notify_test ;

TRUNCATE **subject** ;

INSERT INTO notify_test VALUES('s01', 'Math', 3) ;

NOTICE: Hi, I got notify_trigger invoked FOR ROW AFTER INSERT on **subject**
INSERT 0 1

Truncate command

```
CREATE TRIGGER notify_truncate_trigger
  AFTER TRUNCATE
  ON subject
  FOR EACH STATEMENT
  EXECUTE PROCEDURE func_notify_trigger() ;
```

call:

```
TRUNCATE subject;
```

result:

```
NOTICE: Hi, I got notify_truncate_trigger invoked FOR
STATEMENT AFTER TRUNCATE on subject

TRUNCATE TABLE
```

Audit Trigger

Log data changes to tables.

```
CREATE TABLE audit_log (
  username text, - - who did the change
  event_time_utc timestamp, - - when the event was recorded
  table_name text, - - contains schema-qualified table name
  operation text, - - INSERT, UPDATE, DELETE or TRUNCATE
  before_value json, - - the OLD tuple value
  after_value json - - the NEW tuple value
) ;
```

e.g.

```
CREATE OR REPLACE FUNCTION func_audit_trigger()
RETURNS trigger
LANGUAGE plpgsql AS $$
DECLARE
  old_row json := NULL;
  new_row json := NULL;
BEGIN
  IF TG_OP IN ('UPDATE','DELETE') THEN
    old_row = row_to_json(OLD);
  END IF;
  IF TG_OP IN ('INSERT','UPDATE') THEN
    new_row = row_to_json(NEW);
  END IF;
...
```

e.g.

```
    INSERT INTO audit_log
    VALUES (
      session_user,
      current_timestamp AT TIME ZONE 'UTC',
      TG_TABLE_SCHEMA || '.' || TG_TABLE_NAME,
      TG_OP,
      old_row,
      new_row
    );
    RETURN NEW;
END;
$$
```

e.g.

```
CREATE TRIGGER audit_log
  AFTER INSERT OR UPDATE OR DELETE
  ON subject
  FOR EACH ROW
  EXECUTE PROCEDURE func_audit_trigger();
```

Disallowing DELETE

```
CREATE OR REPLACE FUNCTION cancel_op()
RETURNS TRIGGER
LANGUAGE plpgsql AS $$
BEGIN
  IF TG_WHEN = 'AFTER' THEN
    RAISE EXCEPTION 'YOU ARE NOT ALLOWED TO %
ROWS IN %.%',
       TG_OP, TG_TABLE_SCHEMA, TG_TABLE_NAME;
  END IF;
  RAISE NOTICE '% ON ROWS IN %.% WILL NOT HAPPEN',
       TG_OP, TG_TABLE_SCHEMA, TG_TABLE_NAME;
  RETURN NULL;
END;
$$
```

Disallowing DELETE

Trigger function can be use:
BEFORE: the operation is skipped with a message.
AFTER: ERROR is raised and the transaction is rolled back.
e.g.

```
CREATE TRIGGER disallow_delete
  AFTER DELETE
  ON student
  FOR EACH ROW
  EXECUTE PROCEDURE func_cancel_op();
```

- - -

```
CREATE TRIGGER skip_delete
  BEFORE DELETE
  ON student
  FOR EACH ROW
  EXECUTE PROCEDURE func_cancel_op();
```

Disallowing TRUNCATE

Create an **AFTER** trigger using the same function as for DELETE:

```
CREATE TRIGGER disallow_truncate
  AFTER TRUNCATE ON student
  FOR EACH STATEMENT
  EXECUTE PROCEDURE func_cancel_op();
```

call:

```
TRUNCATE student;
```

result:

ERROR: YOU ARE NOT ALLOWED TO TRUNCATE ROWS IN **public.student**

Disallowing TRUNCATE

Create an **AFTER** trigger using the same function as for DELETE:

```
CREATE TRIGGER disallow_truncate
  AFTER TRUNCATE ON student
  FOR EACH STATEMENT
  EXECUTE PROCEDURE func_cancel_op();
```

call:

```
TRUNCATE student;
```

result:

```
ERROR: YOU ARE NOT ALLOWED TO TRUNCATE ROWS IN
public.student
```

Conditional Triggers

CREATE OR REPLACE TRIGGER **trigger-name**

{ BEFORE | AFTER | INSTEAD OF }

{ **event** [OR **event** ...] }

[OF **column-name** [, **column-name** ...]]

ON **table-name**

[FOR [EACH] { ROW | STATEMENT }]

[**WHEN** (**condition**)]

EXECUTE PROCEDURE **function-name** (arguments)

Integrity Constraints Form

function:

create or replace function **func-name()**
returns trigger language plpgsql as
$$
begin
  if **Condition of Constraint Integrity is invalid** then
    raise notice '%', TG_ARGV[0] ;
    or **raise exception '%', TG_ARGV[0]** ;

    return null;
  end if ;
  return new ; (if event is insert or update)
end ;
$$;

notice: function for trigger is must not have arguments.
argument is sent in the trigger.

Integrity Constraints Form

trigger:

create or replace trigger **trigger-name**

{ before | after | instead of }

{event [or event ...]}

{[of column-name [, column-name ...]] } on **table-name**

for each row

execute procedure **fucn-name**(**'message'**);

e.g. Insert studentgrades

function:

```
create or replace function func_insert_grade()
returns trigger language plpgsql as
$$
begin
  if (new.grades < 0 or new.grades > 10) then
    raise notice '%',TG_ARGV[0] ;
    return null ;
  end if;    return new ;
end ;
$$ ;
—
```

e.g. Insert studentgrades

trigger:

```
create or replace trigger insert_grades
   before insert on studentgrades
   for each row
   execute procedure fnc_insert_grade('grades out of [0,10]') ;
```

—

notice: Trigger send a message.

e.g. Update grades

function:

```
create or replace function func_update_grade()
returns trigger language plpgsql as
$$
begin
  if (new.grades < 0 or new.grades > 10) then
    raise notice '%',TG_ARGV[0] ;
    return null ;
  end if;
  return new ;
end ;
$$ ;
—
```

notice: this function is the same insert function.

e.g. Update grades

trigger:

create or replace trigger **update_grades**
before update of grades on studentgrades
for each row
execute procedure **func_update_grade('grades update out of [0,10]')** ;

—

notice: in this case, we can call insert function.

$R(ABC)$, $A \rightarrow B$: integrity constraints

context: $R(ABC)$

condition:

$$\forall (t_1, t_2) \in \forall r(R) : t_1.A = t_2.A$$

$$t_1.B = t_2.B$$

end.

influence table:

	insert	**delete**	**update**
$R(ABC)$	$+$	-	$+(A/B)$

$R(ABC)$, $A \rightarrow B$, **func: insert** R

```
create or replace function func_R()
returns trigger language plpgsql as
$$
Begin
  if (select count(distinct B)
      from R
      where new.A = A ) > 1 then
     raise exception '%', TG_ARGV[0];
     return null;
  end if;
  return new;
end;
$$
```

$R(ABC)$, $A \to B$, **trigger: insert** R

create or replace trigger **insert_R**
after insert on R
for each row
execute procedure **func_R('insert invalid** $A \to B$**')**;

e.g.

R	A	B	C
	1	1	1
insert valid	1	1	2
insert invalid	1	2	3

$R(ABC)$, $A \to B$, trigger: update A or B of R

create or replace trigger **update_R**
after update of A , B on R
for each row
execute procedure **func_R('update invalid $A \to B$')**;

e.g.

R	A	B	C
	1	1	1
update A valid: 2 –> 4	2	2	2
update A invalid: 3 –> 1	3	3	3

R	A	B	C
	1	1	1
update B valid: 2 –> 1	2	2	2
update B invalid: 1 –> 2	1	1	3

$R1(ABC), R2(CDE)$, $B \to D$: integrity constraints

context: $R1(ABC), R2(CDE)$

condition:

$$\forall (t_1, t_2) \in \forall r(R1 \bowtie R2) : t_1.B = t_2.B$$

$$t_1.D = t_2.D$$

end.

influence table:

	insert	**delete**	**update**
$R1(ABC)$	$+$	-	$+(B/C)$
$R2(CDE)$	$+$	-	$+(C/D)$

$R1(ABC), R2(CDE)$, $B \to D$, **func: insert** $R1$

```
create or replace function func_R1()
returns trigger language plpgsql as
$$
Begin
  if ( select count(distinct D)
       from R1, R2
       where R1.B = new.B and R1.C = R2.C ) > 1 then
     raise exception '%', TG_ARGV[0] ;
     return null;
  end if;
  return new;
end;
$$
```

$R1(ABC), R2(CDE)$, $B \rightarrow D$, **trigger: insert** $R1$

create or replace trigger **insert_R1**

after insert on R1

for each row

execute procedure **func_R1('insert invalid B -> D')** ;

e.g.

R1	(A	B	C)	R2	(C	D	E)
	1	1	1		1	1	1
					2	2	2
insert valid	2	2	2				
insert invalid	3	1	2				

$R1(ABC), R2(CDE)$, $B \rightarrow D$, trigger: update B or C

create or replace trigger **update_B_C_R1**

after update of B , C on R1

for each row

execute procedure **func_R1('update invalid B –> D')** ;

e.g.

R1	(A	B	C)	R2	(C	D	E)
	1	1	1		1	1	1
update B valid: 2 –> 3	2	2	2		2	2	2
update B invalid: 2 –> 1							
update C valid: 3 –> 4	3	1	3				
update C invalid: 3 –> 2							

$R1(ABC), R2(CDE)$, $B \rightarrow D$, **func: insert** $R2$

```
create or replace function func_R2()
returns trigger language plpgsql as
$$
Begin
  if exists ( select B
       from R1, R2
       where R1.C = R2.C
       group by B
       having count(*) > 1 ) then
     raise exception '%', TG_ARGV[0] ;
     return null;
  end if;
  return new;
end;
$$
```

$R1(ABC), R2(CDE)$, $B \rightarrow D$, **trigger: insert** $R2$

create or replace trigger **insert_R2**

after insert on R2

for each row

execute procedure **func_R2('insert invalid B –> D')** ;

e.g.

R1	(A	B	C)	R2	(C	D	E)
	1	1	1		1	1	1
insert valid					2	2	2
insert invalid					1	2	3

$R1(ABC), R2(CDE)$, $B \rightarrow D$, trigger: update C or D

create or replace trigger **update_C_D_R2**

after update of C , D on R2

for each row

execute procedure **func_R2('update invalid B –> D')** ;

e.g.

R1	(A	B	C)	R2	(C	D	E)
	1	1	1		1	1	1
update C valid: 2 –> 3					2	2	2
update C invalid: 2 –> 1							
update D valid: 2 –> 3							
update D invalid: 1 –> 2					1	1	3

Foreign Key

[CONSTRAINT **fk-name**]

FOREIGN KEY (**foreign-key**)

REFERENCES **parent-table** (**parent-key**)

[ON DELETE **delete-action**]

[ON UPDATE **update-action**]

Delete and Update actions: when the primary key in the parent table is deleted and updated. Primary key is rarely updated.
Action:

- SET NULL
- SET DEFAULT
- RESTRICT
- NO ACTION
- CASCADE

e.g. foreign key

```
create table class (
  classID varchar(3) primary key,
  ...
)

create table student (
  studentID varchar(3) primary key,
  ...
  classID varchar(3),
  constraint fkClass foreign key (classID)
    references class (classID)
    on delete set null
)
```

on update or delete: Action default is **no action**.

Create index

CREATE INDEX **indexName** ON **tableName** [**USING method**]
(
  columnName [**ASC** | **DESC**] [**NULLS** { **FIRST** | **LAST** }],
  ...
);

method: btree, hash, gist, spgist, gin, and brin.
PostgreSQL uses **btree** by default.
ASC, DESC: sort order.
NULLS FIRST is the default when **DESC**.
NULLS LAST is the default when **DESC**.

Drop index

DROP INDEX [**CONCURRENTLY**]
[**IF EXISTS**] **indexName**
[**CASCADE** | **RESTRICT**];

IF EXISTS: Attempting to remove a non-existent index will result in an error.
CASCADE: Automatically drop all objects that depends on index.
RESTRICT: Refuse to drop the index if any objects depend on.
CONCURRENTLY: Waits until the conflicting transaction completes before removing the index.

Index types

B-tree indexes: Self-balancing tree that maintains sorted data and allows searches, insertions, deletions, and sequential access in logarithmic time.
Hash indexes: Can handle only simple equality comparison (=). It means that whenever an indexed column is involved in a comparison using the equal(=) operator, the query planner will consider using a hash index.
GIN indexes: Useful when multiple values stored in a single column.
BRIN: Block Range indexes. Allows the use of an index on a very large table. BRIN is much smaller and less costly to maintain in comparison with a B-tree index.
GiST Indexes: Generalized Search Tree. Useful in indexing geometric data types and full-text search.
SP-GiST Indexes: Space-Partitioned GiST. Supports partitioned search trees.

UNIQUE index

UNIQUE index enforces the uniqueness of values in one or multiple columns.

CREATE **UNIQUE** INDEX **indexName**
ON **tableName** (**columnName**, [...]);

index on expression

CREATE INDEX **indexName**
ON **tableName** (**expression**);

e.g.

CREATE INDEX **idxStudentName**
ON student(**LOWER**(**studentName**));

Multicolumn indexes

CREATE INDEX **indexName**
ON **tableName** (**a, b, c, ...**);

When defining a multicolumn index, you should place the columns which are often used In the WHERE clause at the beginning of the column list and the columns that are less frequently used in the condition after.

e.g.
WHERE a = v1 and b = v2 and c = v3;

e.g.
WHERE a = v1 and b = v2;

e.g.
WHERE a = v1 ;